Theory of Concurrency

Lecture 6: Mazurkiewicz traces and Zielonka automata

Piotr Hofman

Plan

1. Problems with linear semantics.
2. Histories.
3. Independence relations and Mazurkiewicz traces.
4. Dependence graphs, partial orders and Foata normal form.
5. Trace-closed languages and their specifications.
6. Zielonka automata and the distributed realisation theorem.

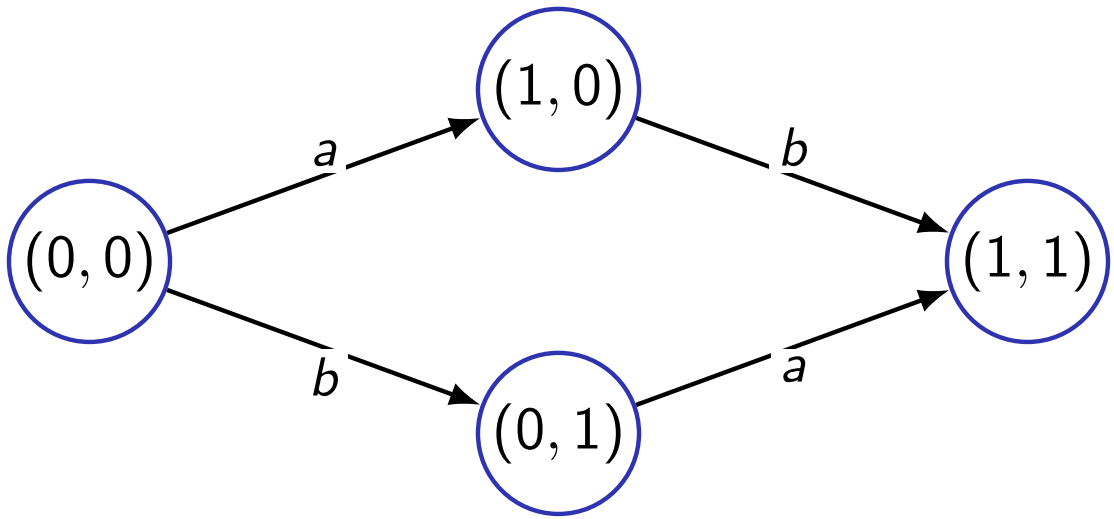
Central question

How can we describe and implement concurrent behaviour without fixing an arbitrary order between independent events?

Two independent actions in an LTS

Two local variables

Initially $(x, y) = (0, 0)$. Action a sets $x := 1$ and action b sets $y := 1$.



Two paths, the same result

Both ab and ba are executions, and both end in $(1, 1)$.

What does an ordinary trace record?

Interleaving semantics

A run of an LTS produces a word: a *total order* of the actions executed.

$$ab \neq ba \qquad \text{as words in } \Sigma^*.$$

But in the preceding example

Neither action affects the other one's enabledness or result. The difference between ab and ba is only their scheduling order.

We want to retain necessary order, but forget irrelevant order.

Interleaving explosion

Two sequential processes

One process performs $a_1, a_2, \ldots, a_m$ in order; another performs $b_1, b_2, \ldots, b_n$ in order. Cross-process actions are independent.

$$\text{number of interleavings} = \binom{m+n}{m}.$$

First process	Second process	Interleavings
$m = 2$	$n = 2$	6
$m = 4$	$n = 4$	70
$m = 10$	$n = 10$	184 756

Fairness and linearisation

Two concurrent loops

Consider one process that repeatedly emits a, and another that repeatedly emits b.

$$P_1 : a\,a\,a\,\ldots \qquad || \qquad P_2 : b\,b\,b\,\ldots$$

These are independent threads of activity, not a single sequential program.

A natural fairness requirement

The specification

$$\mathbf{G}(a \Rightarrow \mathbf{F}b) \text{does not hold for the linearisation } \; a, a, a, \cdots .$$

Why this matters

Fairness is a property of the run as a concurrent system, not of an arbitrary total order chosen by the scheduler. Linear semantics can therefore hide behavioural defects or impose artificial ordering constraints.

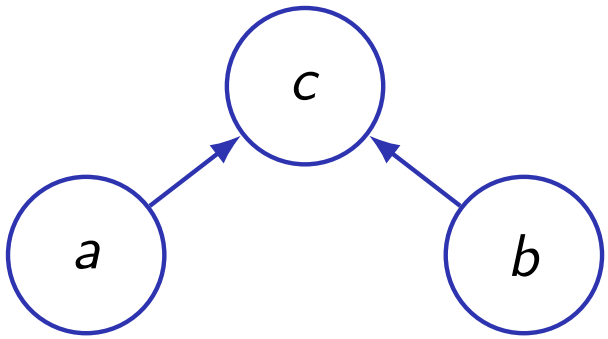
Causality

Independence versus dependence

Two events are *independent* if neither one enables, disables, nor changes the effect of the other. They may be executed in either order without changing the observable outcome.

Causal order

If one event is required to occur before another because it influences its result or enables it, then the first is a *cause* of the second. This gives a partial order, not a total one.



same result regardless of *abc* or *bac*

Why linear semantics is not enough

A scheduler chooses an order, but not necessarily a causal one

An execution of a concurrent system is often represented as a word

$$a_1 a_2 \cdots a_n.$$

Problems

- The word order records arbitrary scheduling choices, not essential causality.
- Fairness may require that a process is eventually scheduled, even if its actions are not the next ones in a fixed linearisation.
- Exponential blow-up appears because independent actions generate many equivalent serialisations.

The semantics should keep only the order that matters for behaviour.

Distributed logs (Another problem)

Problem with logging

In distributed systems, each machine writes its local events with its own clock. A global log is often assembled later by merging timestamps, but different clocks may drift and messages may be delayed or reordered.

Example: Patriot missile failure

- After 100 hours, the system clock drifted by 0.34 seconds.
- This small error shifted the tracked position of a Scud by roughly 500 meters.
- The missile missed its target; 28 soldiers were killed.

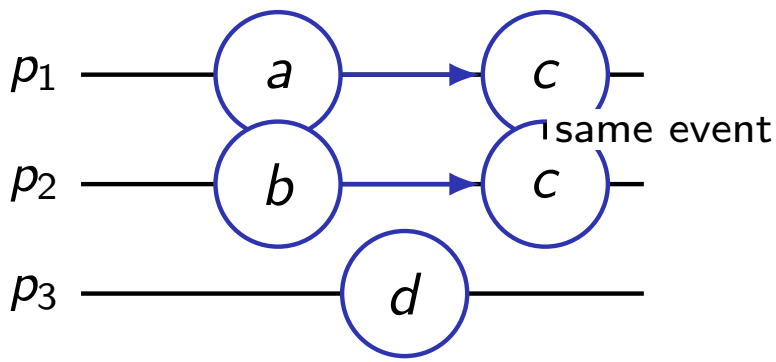
Histories

Processes and their alphabets

Let $P = \{p_1, p_2, p_3\}$, with local alphabets

$$\Sigma_{p_1} = \{a, c\}, \quad \Sigma_{p_2} = \{b, c\}, \quad \Sigma_{p_3} = \{d\}.$$

An action shared by processes is communication: they synchronize by executing that same action. Here c is one event involving both p_1 and p_2.



Histories (Approach 1)

What should be dependnet?

Actions in the same process are dependent. For them, we have causality relation.

Dependence and independence

Actions are dependent iff their process domains intersect; disjoint domains mean independence. Thus a and b are independent, as is d from a, b, c.

Independence and dependence alphabets (Approach 2)

Definition

A *concurrent alphabet* is (Σ, I), where Σ is finite and

$$I \subseteq \Sigma \times \Sigma$$

is a **symmetric, irreflexive** independence relation.

Dependence relation

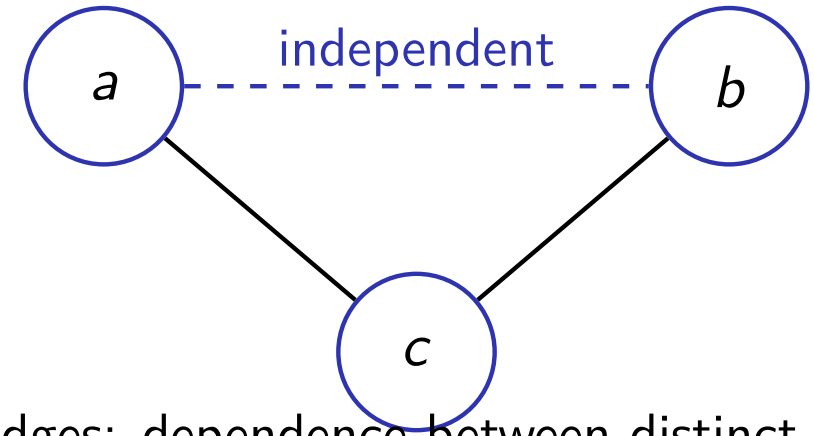
$$D = (\Sigma \times \Sigma) \setminus I.$$

Dependence is symmetric and reflexive.

We read $a\ I\ b$ as "a and b may commute", and $a\ D\ b$ as "their order must be retained".

A running dependence alphabet

$$\Sigma = \{a, b, c\}, \qquad I = \{(a, b), (b, a)\}.$$



solid edges: dependence between distinct actions

Important distinction

$a \; I \; b$, while $a \; D \; c$ and $b \; D \; c$. Dependence need not be transitive; every action also depends on itself.

Swapping independent neighbours

Elementary equivalence

If $a \; I \; b$, then for every $u, v \in \Sigma^*$,

$$uabv \longleftrightarrow ubav.$$

Let $\equiv_I$ be the equivalence relation generated by these swaps.

Example: $I = \{(a, b), (b, a)\}$

$$\underbrace{abc \equiv_I bac}_{\text{swap } a,b}, \qquad \underbrace{abc \not\equiv_I acb}_{\text{cannot swap } b,c}.$$

Swaps act on *adjacent* positions; a sequence of swaps may be needed.

Mazurkiewicz traces

Definition

A *Mazurkiewicz trace* is an equivalence class of words under $\equiv_I$:

$$[w]_I = \{v \in \Sigma^* : v \equiv_I w\}.$$

Our example

For $I = \{(a, b), (b, a)\}$,

$$[abc]_I = \{abc, bac\}, \qquad [acb]_I = \{acb\}.$$

A trace represents an execution *up to permutations of independent actions*.

Why traces form a monoid

Congruence property

Swapping independent neighbours remains valid inside larger words:

$$u \equiv_I u', \quad v \equiv_I v' \quad \Longrightarrow \quad uv \equiv_I u'v'.$$

Trace monoid

$$M(\Sigma, I) = \Sigma^*/\equiv_I, \qquad [u]_I \cdot [v]_I = [uv]_I.$$

The empty word ε induces the identity element $[\varepsilon]_I$.

Concatenation of traces is well-defined, independently of the chosen word representatives.

Two extreme forms of independence

No independent distinct actions: $I = \varnothing$

No swaps are allowed. Every trace consists of one word:

$$M(\Sigma, \varnothing) = \Sigma^*.$$

All distinct actions independent

$I = \{(a, b) : a \neq b\}$. Words are equivalent exactly when each letter occurs equally often.

$$abc \equiv_I bca \equiv_I cab.$$

The trace remembers only the numbers of occurrences of actions.

Most concurrent alphabets lie between these extremes.

By $u \div a$ we mean right cancellation of a from u. For u beeing a string it is defined as follows:

$$u \div a = \begin{cases} \varepsilon \text{ for } u = \varepsilon, \\ v \text{ for } u = va \\ (v \div a)b \text{ for } u = vb \text{ where } b \neq a \end{cases}$$

Lemma

$u \equiv w \implies u \div a \equiv w \div a$ for any $a \in \Sigma$.

By $u \div a$ we mean right cancellation of a from u. For u beeing a string it is defined as follows:

$$u \div a = \begin{cases} \varepsilon \text{ for } u = \varepsilon, \\ v \text{ for } u = va \\ (v \div a)b \text{ for } u = vb \text{ where } b \neq a \end{cases}$$

Lemma

$u \equiv w \implies u \div a \equiv w \div a$ for any $a \in \Sigma$.

A dependency morphism w.r.t. D is any homomorphism from the monoid of strings over Σ on to another monoid such that

- A1 $\phi(w) = \phi(\varepsilon) \implies w = \varepsilon$;
- A2 $(a, b) \in I \implies \phi(ab) = \phi(ba)$;
- A3 $\phi(ua) = \phi(v) \implies \phi(u) = \phi(v \div a)$;
- A4 $\phi(ua) = \phi(vb) \wedge a \neq b \implies (a, b) \in I$

The dependence morphism

Pairwise projections

For each dependent pair $a \; D \; b$, let $\pi_{a,b}(w)$ erase all letters except a and b (for $a = b$, retain only a). Define

$$\delta : \Sigma^* \longrightarrow \prod_{(a,b)\in D} \{a, b\}^*, \qquad \delta(w) = \left(\pi_{a,b}(w)\right)_{(a,b)\in D}.$$

Why it is a trace morphism

Each projection preserves concatenation, and swapping adjacent independent letters changes none of the projections. Hence δ factors through the trace monoid $M(\Sigma, I)$ and records the order of every dependent pair of action labels.

Projection lemma

Lemma: traces are uniquely determined by projections

For all $u, v \in \Sigma^*$,

$$\delta(u) = \delta(v) \quad \Longleftrightarrow \quad [u]_I = [v]_I.$$

Equivalently, the induced dependence morphism on $M(\Sigma, I)$ is injective: its image identifies a unique trace.

One direction

If $u \equiv_I v$, they differ by swaps of adjacent independent letters. No such swap changes the projection onto any dependent pair, so $\delta(u) = \delta(v)$.

Proof of the projection lemma

Move the last letter

Suppose $\delta(u) = \delta(v)$ and write $u = u'a$. The projection onto each pair (a, b) with $a \; D \; b$ shows that no dependent letter occurs after the last a in v. Therefore that last a can be swapped rightward across the suffix, giving $v \equiv_I v'a$.

Cancel and use induction

The projections of u' and v' agree: for pairs involving a, remove the common final a; all other projections are unchanged. Induction on word length gives $[u']_I = [v']_I$. Right cancellation (or appending a) yields $[u]_I = [v]_I$.

Proof of trace-history equivalence

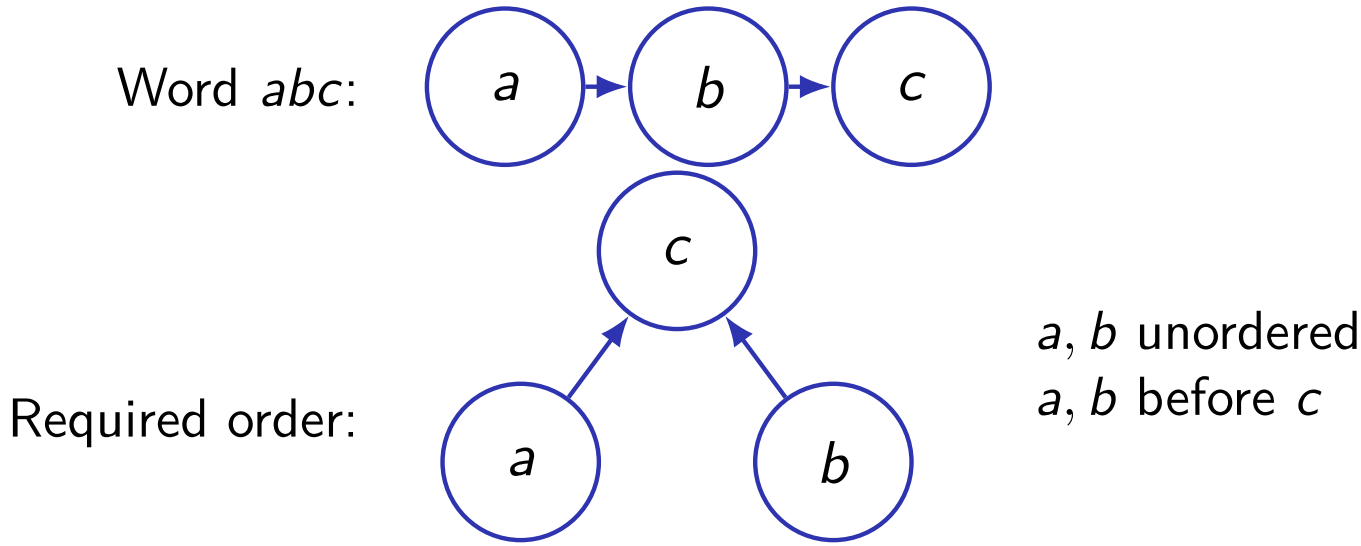
Histories determine projections

In a history, occurrences with the same label form a chain. For each dependent pair $a \; D \; b$, restricting the history to its a- and b-events gives a total order whose labels spell exactly $\pi_{a,b}(w)$. Thus $H(u) \cong H(v)$ implies $\delta(u) = \delta(v)$.

Projections determine histories

Conversely, equal projections match the kth occurrence of each label and give the same order for every dependent pair of events. Their transitive closures therefore give isomorphic labelled histories. By the projection lemma, equal projections are equivalent to $u \equiv_I v$, proving the theorem.

From total orders to partial orders



Idea

A word imposes an order on *all* positions. A dependence graph keeps only the order forced by dependent actions.

Dependence graph of a word

Construction

Let $w = a_1 a_2 \cdots a_n$. Create a distinct event vertex for each position: Add an edge $i \to j$ whenever

$$i < j \quad \text{and} \quad (a_i, a_j) \in D.$$

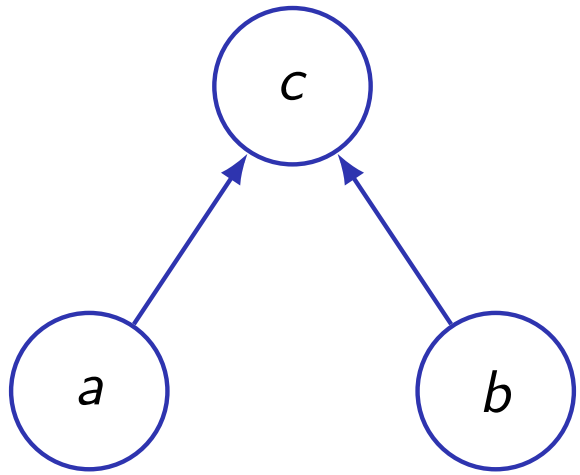
Take the reflexive transitive closure to obtain the order $\leq_w$.

Important

Vertices are **occurrences**, not action names. In particular, two occurrences of the same action are ordered because $(a, a) \in D$.

Dependence graph: *abc*

$$I = \{(a, b), (b, a)\}, \qquad w = abc.$$



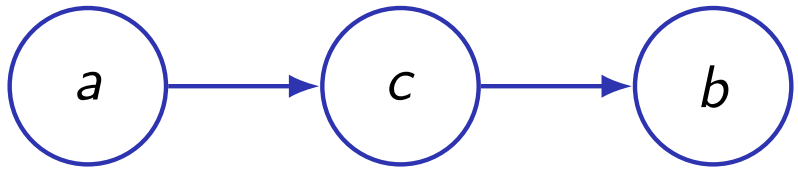
Linearizations

The labelled partial order has exactly two linearizations:

$$\mathrm{Lin}(E_w, \leq_w, \lambda) = \{abc, bac\}.$$

Independent labels can still be ordered

$$I = \{(a, b), (b, a)\}, \qquad w = acb.$$



Transitive causality

Although $a \; I \; b$, the order contains $a < c < b$ because both actions depend on c.

Independence of *action labels* does not guarantee that two particular *occurrences* are unordered.

Fundamental characterisation of traces

Theorem

For finite words $u, v \in \Sigma^*$,

$$u \equiv_I v \quad \Longleftrightarrow \quad \text{their labelled dependence posets are isomorphic.}$$

Proof idea: $\Rightarrow$

Swapping adjacent independent events preserves every order constraint between dependent occurrences.

Proof idea: $\Leftarrow$

Any two linear extensions of a finite poset are connected by adjacent swaps of incomparable events. In a dependence poset, incomparable events have independent labels.

Lexicographic normal form

Choose an order on the alphabet

Fix a total order, for example $a < b < c$. Each trace contains finitely many words, so it has a unique lexicographically least representative.

$$\operatorname{lexmin}([w]_I) = \min_{\text{lex}}\{v \in \Sigma^* : v \equiv_I w\}.$$

Example

With $I = \{(a, b), (b, a)\}$,

$$[abc]_I = \{abc, bac\}, \qquad \operatorname{lexmin}([abc]_I) = abc.$$

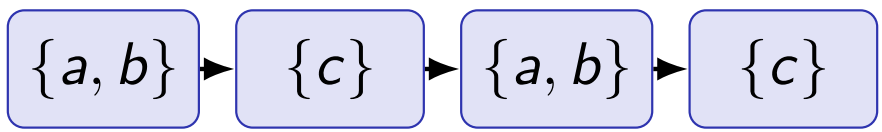
The chosen word is canonical, but it is still a *total order*: it does not display which events could have happened concurrently.

Foata normal form: concurrent layers

Canonical representation

Repeatedly take all minimal events of the dependence poset, record them as one layer, and remove them. Every layer consists of pairwise independent actions.

$$w = abcabc, \qquad I = \{(a, b), (b, a)\}.$$



Foata normal form

$$\{a, b\}\ \{c\}\ \{a, b\}\ \{c\}.$$

These layers are uniquely determined by a finite trace.

Foata layers are not time slots

$$\Sigma = \{a, b, c\}, \qquad I = \{(a, b), (b, a), (b, c), (c, b)\}.$$

Thus $a \; D \; c$, while b is independent of both a and c.

One trace, two linearizations

The words abc and acb differ only by swapping the independent actions b, c. Their dependence order has just one constraint: $a < c$.

$$\text{Foata}(abc) = \text{Foata}(acb) = \{a, b\}\ \{c\}.$$

How to read the layers

The layers give events their dependence depth; they are not strict time slots. Here b is in the first layer but may occur after c, as the linearization acb shows.

Foata form and lexicographic form

	Lexicographic	**Foata**
Representation	One chosen word	A sequence of concurrent layers
Captures concurrency	No; serialises a trace	Yes; independent events can share a layer
Canonical?	Yes, after fixing an alphabet order	Yes, from the dependence order
Useful for	Comparing or storing traces as words	Seeing causal depth and available parallelism

Same trace, different emphasis

For $[abc]_I = \{abc, bac\}$ with $a \; I \; b$, the lexicographic form is abc, while the Foata form is $\{a, b\}\{c\}$.

What information does a trace preserve?

Preserved

Event occurrences, action labels, and the ordering constraints induced by dependence.

Forgotten

The chosen interleaving of incomparable events. A trace does not record physical durations or a schedule.

Not represented by one trace

Branching choices between different possible executions of an entire system.

From word languages to trace languages

Quotient map

$$q : \Sigma^* \longrightarrow M(\Sigma, I), \qquad q(w) = [w]_I.$$

A word language $L \subseteq \Sigma^*$ induces the trace language

$$q(L) = \{[w]_I : w \in L\}.$$

What can go wrong?

If L contains only some linearizations of a trace, the quotient forgets which representatives belonged to L.

Trace-closed languages

Definition

A language $L \subseteq \Sigma^*$ is *I-closed* (or *trace-closed*) if

$$u \in L \text{ and } u \equiv_I v \quad \Longrightarrow \quad v \in L.$$

Equivalently, $uabv \in L$ iff $ubav \in L$ whenever $a \; I \; b$.

Example

Let $\Sigma = \{a, b, c\}$, with $a \; I \; b$ and both dependent on c. Let

$$A = (a + b)^*, \qquad R = AaAbA + AbAaA, \qquad L = (Rc)^* A.$$

R consists of words containing both letters; each c closes one such block. Swapping neighbouring a, b preserves membership in L.

A requirement that is not trace-closed

$$\Sigma = \{a, b\}, \qquad a \; I \; b, \qquad L = \{ab\}.$$

Problem

$ab \in L$, but $ba \notin L$, although $ab \equiv_I ba$.

$$q^{-1}(q(L)) = \{ab, ba\} \neq L.$$

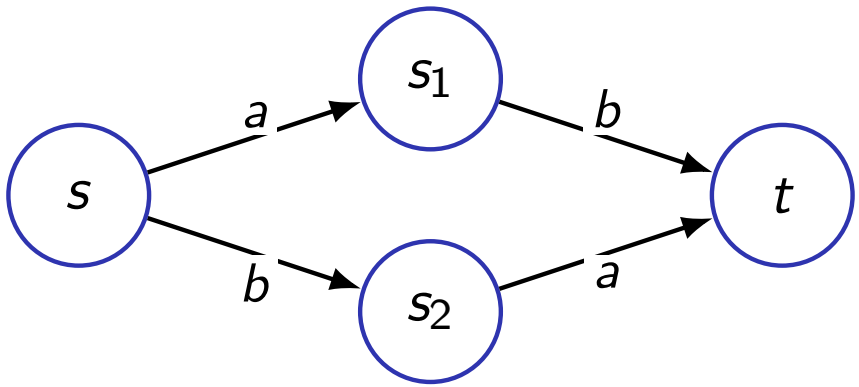
Interpretation

The specification explicitly requires a before b, so these two actions cannot simultaneously be treated as independent for this requirement.

Why an LTS should satisfy diamonds

Commutation property

For independent a, b, whenever $s \xrightarrow{a} s_1 \xrightarrow{b} t$, there must be some s_2 with $s \xrightarrow{b} s_2 \xrightarrow{a} t$.



Consequence

If this replacement is always possible along runs, the finite trace language of the LTS is I-closed.

From regular specifications to distributed machines

Finite-state specification

Regularity means that a finite automaton can recognise the word language $L \subseteq \Sigma^*$. This is a finite-state description of the allowed executions.

Concurrency-respecting specification

If L is I-closed, acceptance depends on the trace, not on which order independent actions happened to take.

Implementation question

Can such a specification be implemented with finite local states, so each action accesses only the processes that participate in it?

Zielonka's theorem says yes for every regular, I-closed language.

Distributed alphabets

Definition

Let P be a finite set of processes. A *distributed alphabet* provides

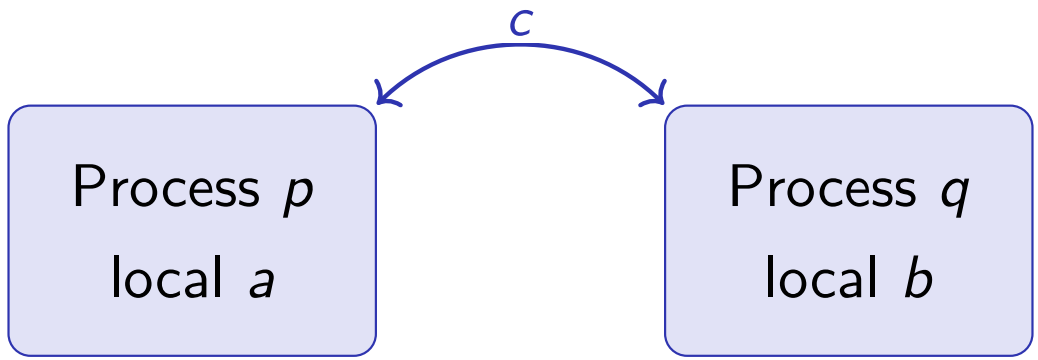
$$\mathrm{dom} : \Sigma \longrightarrow 2^P \setminus \{\varnothing\}.$$

The set $\mathrm{dom}(a)$ contains exactly the processes that participate in a.

action	domain	meaning
a	$\{p\}$	local action of p
b	$\{q\}$	local action of q
c	$\{p, q\}$	synchronization of p and q

$$a \; I \; b \quad \Longleftrightarrow \quad \mathrm{dom}(a) \cap \mathrm{dom}(b) = \varnothing.$$

Why process domains explain independence



Local and shared actions

a changes only the state of p, while b changes only the state of q. Action c involves both processes and can coordinate their states.

Disjoint process domains give a structural reason for the commutation of actions.

Zielonka automata: local states

Global configuration

Each process $p \in P$ has a finite local state space Q_p and initial state q_p^0. A global configuration is a tuple

$$(q_p)_{p \in P} \in \prod_{p \in P} Q_p.$$

Action semantics

An action a reads and updates *only* the states of processes in $\mathrm{dom}(a)$. All other local states remain unchanged.

A shared action is one atomic synchronization step of all its participating processes.

Formal definition of a Zielonka automaton

Deterministic finite asynchronous automaton

A Zielonka automaton consists of finite local state sets $(Q_p)_{p\in P}$, an initial tuple q^0, a set of accepting global states

$$F \subseteq \prod_{p\in P} Q_p,$$

and, for each $a \in \Sigma$, a *partial* local update function

$$\delta_a : \prod_{p\in \mathrm{dom}(a)} Q_p \rightharpoonup \prod_{p\in \mathrm{dom}(a)} Q_p.$$

Reading a word

Execute its actions from q^0 when their local updates are defined. A word is accepted if the final global tuple belongs to F.

A two-process Zielonka automaton

Local states and initial configuration

Processes p and q each have local state space $\{0, 1\}$, with initial state 0. Thus the initial global configuration is $(0, 0)$.

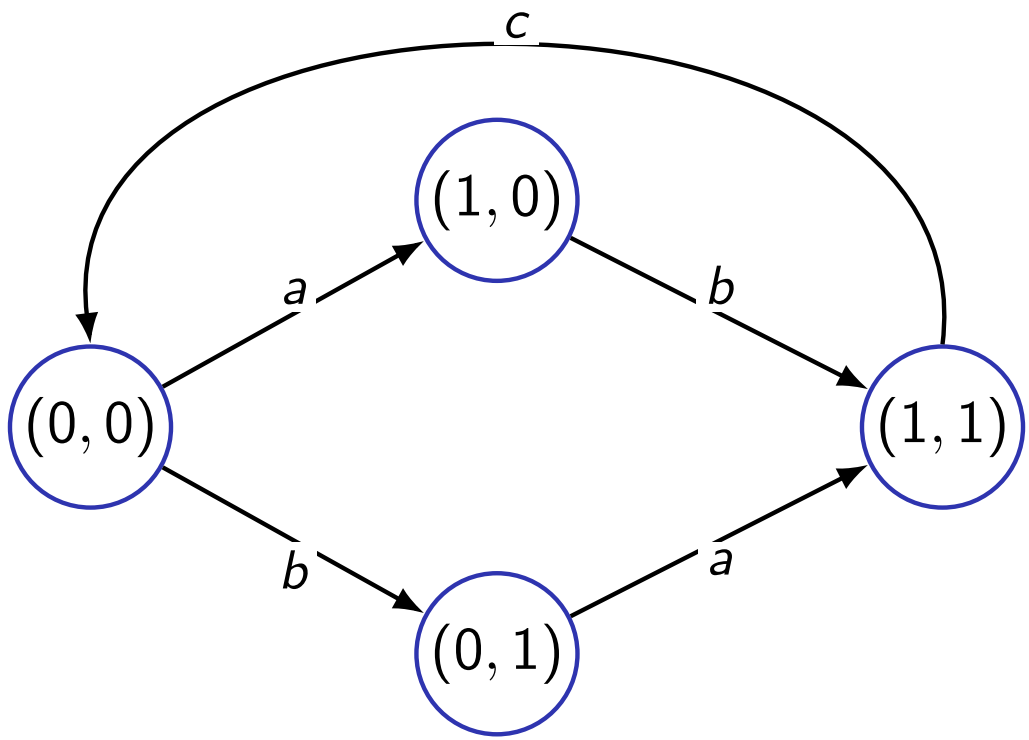
action	domain	partial local update
a	$\{p\}$	$0 \mapsto 1$ on p
b	$\{q\}$	$0 \mapsto 1$ on q
c	$\{p, q\}$	$(1, 1) \mapsto (0, 0)$

Reading the updates

Actions a and b are enabled only when their process is in state 0; they are independent. The synchronization c is enabled only when both processes are in state 1 and resets both.

Every global state is accepting. The next slide shows the resulting global LTS.

Global LTS of the Zielonka automaton



Global view, local implementation

The diagram is an ordinary LTS, but its transitions are generated from local updates and one shared synchronization.

The language recognised by the example

Accept all reachable global states

Take $F = Q_p \times Q_q$. Then the accepted language consists of valid prefixes of rounds with one a, one b, and a closing c:

$$L = \mathrm{Pref}\big(((ab + ba)c)^*\big).$$

accepted	not accepted
$\varepsilon,\ a,\ b,\ ab,\ ba$	c
$abc,\ bac,\ abcabc$	$acb,\ aabc$

Trace closure

Every exchange of adjacent a, b preserves acceptance: abc and bac are two linearizations of one accepted trace.

Independence is built into the automaton

Disjoint local updates commute

If $\mathrm{dom}(a) \cap \mathrm{dom}(b) = \varnothing$, the two updates read and modify disjoint coordinates.

$$q \xrightarrow{a} q_1 \xrightarrow{b} q' \quad \Longleftrightarrow \quad q \xrightarrow{b} q_2 \xrightarrow{a} q'.$$

Consequence

Every word accepted by a Zielonka automaton can be replaced by any I-equivalent word without changing the final global state. Its language is I-closed.

This follows from the architecture, not from a separately imposed commutation axiom.

Zielonka's theorem

Theorem (Zielonka, 1987)

Fix a finite distributed alphabet (Σ, dom) and its induced independence relation I.
If a language $L \subseteq \Sigma^*$ is

- regular, and
- I-closed,

then L is recognised by a **finite deterministic Zielonka automaton** over this distributed alphabet.

Conversely

Every language recognised by a finite Zielonka automaton (with global final states) is regular and I-closed.

What does Zielonka's theorem say?

Specification

A centralised finite automaton can specify a global behaviour language, provided it is closed under the independence relation.

Implementation

There exists a finite automaton with separate local states: every action uses only the local states of its participating processes.

Limit of the result

The theorem is an expressiveness result. It does *not* promise a small distributed implementation. The construction and its size bounds are nontrivial.

Why local projections are not enough

Two processes and a trace-closed language

Let $\mathrm{dom}(a) = \{p\}$, $\mathrm{dom}(b) = \{q\}$, and $\mathrm{dom}(c) = \{p, q\}$, so $a \; I \; b$. Consider

$$L = \{acb,\ bca\}.$$

This language is I-closed: the independent actions a, b are never adjacent in either word.

Project onto each process

$$\pi_p(L) = \{ac, ca\}, \qquad \pi_q(L) = \{cb, bc\}.$$

A construction accepting whenever both local projections are accepted also accepts $w = abc$, since

$$\pi_p(w) = ac \in \pi_p(L), \qquad \pi_q(w) = bc \in \pi_q(L).$$

But $abc \notin L$.

The missing information: correlation

What the projections forget

In acb, process p performs ac while q performs cb. In bca, they perform ca and bc. Separate projections also allow the mixed combination ac with bc, producing the spurious word abc.

Scope of the counterexample

This refutes the rule "accept iff every process accepts its projection," not Zielonka's theorem. In this example, local states can retain which projected word occurred, and a suitable set of *global* accepting tuples can accept the two intended combinations while rejecting the mixed one.

Why the theorem is nontrivial

A distributed automaton must preserve enough information about which local behaviours can occur together. The theorem guarantees a finite construction that does so; simply recognising each projection independently does not.

empty slide

Connection with partial-order reduction

Redundant interleavings

If n distinct actions are pairwise independent, they admit $n!$ different linear orders but form a single Mazurkiewicz trace.

Verification idea

When verifying a property insensitive to independent swaps, it may suffice to explore representatives of trace classes instead of every schedule.

Caution

Partial-order reduction needs conditions ensuring preservation of the chosen property. Trace equivalence alone does not preserve arbitrary ordering-sensitive or branching properties.

Three views of the same system

View	**Information retained**
Word traces	One total order of observed actions
Mazurkiewicz traces	Occurrences and their necessary dependence order
Bisimulation	Branching alternatives and one-step matching

Distinctions

Bisimulation addresses branching. Mazurkiewicz traces address artificial ordering between independent events. These are different kinds of abstraction.

Scope: finite versus infinite executions

In this lecture

Mazurkiewicz traces are equivalence classes of *finite* words, so the trace monoid and finite dependence posets suffice.

Earlier lectures

LTL specifications often concern *infinite* executions. Extensions to infinite traces require appropriate notions of infinite partial orders and careful treatment of properties such as fairness.

A finite trace is an abstraction of a finite execution prefix, not a claim that the system eventually terminates.

Summary

Mazurkiewicz traces

$$a \; I \; b \implies uabv \equiv_I ubav, \qquad [w]_I \in M(\Sigma, I).$$

A trace is represented by a labelled dependence partial order; its words are the linearizations of that order.

Distributed automata

In a Zielonka automaton, a accesses only $\mathrm{dom}(a)$, so disjoint actions commute. Every regular I-closed language admits a finite deterministic Zielonka implementation.

Concurrency appears as the absence of unnecessary ordering.

References



W. Zielonka.
Notes on finite asynchronous automata.
RAIRO – Theoretical Informatics and Applications, 21(2):99–135, 1987.



V. Diekert and G. Rozenberg (eds.).
The Book of Traces.
World Scientific, 1995.



C. Baier and J.-P. Katoen.
Principles of Model Checking.
MIT Press, 2008.